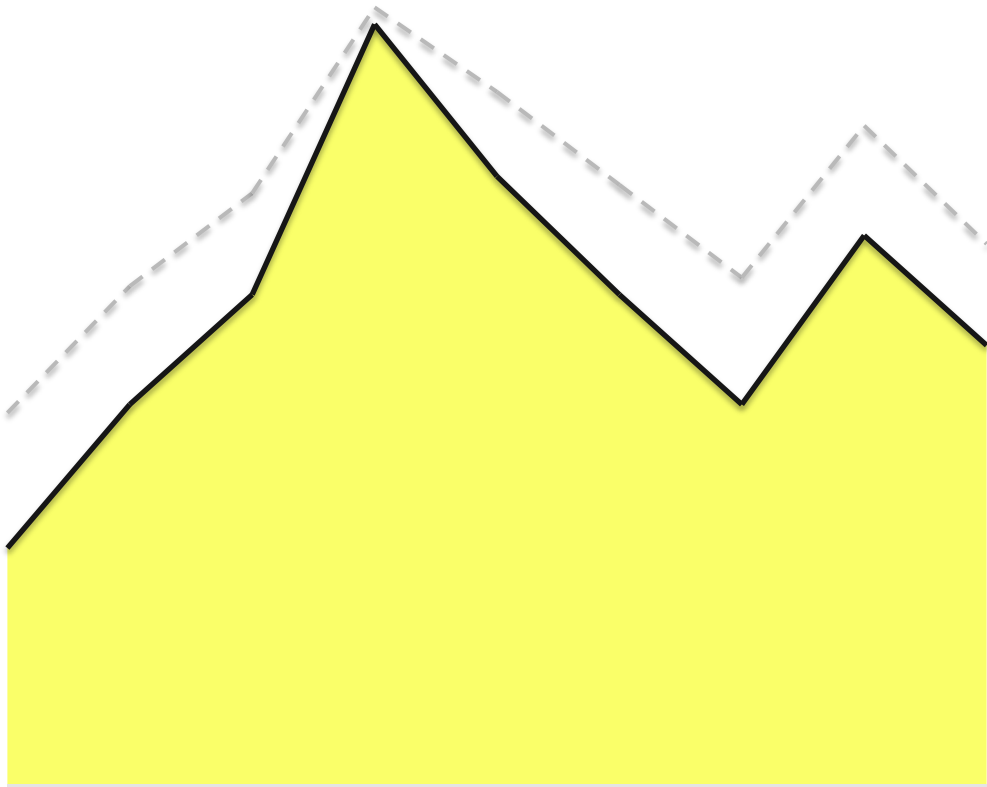




LIVE NOW — clickliv.vercel.app





90%



clickliv.vercel.app

- One day, 7,000,000 real events, plotted minute by minute.
Foreground-only concurrency for 2026-07-31, no mock data.
- The naive curve drawn behind it, same axis.
The 9% / 90% gap is visible on screen, not just claimed in a table.
- Every dataset dimension is a live filter.
Platform, video type, language, app/player version, title, video_resolution — repaints instantly.
- Purpose-built, because no BI tool draws this comparison.
Filters bind as parameters to one ClickHouse view — never concatenated SQL.

WHAT'S UNDERNEATH

- **ClickHouse Cloud, ap-south-1, 2 replicas.**
- **A serverless proxy on Vercel, one parameterized view.**
- **The restricted, budgeted marts_agent role.**
- No admin credential exists in that environment —
the API response says so, in a served_by field.



One dense day, 7,000,000 real events

MEASURE	VALUE
Peak, foreground-only	22,175 at 11:16 UTC
Peak, naive (every open session)	24,196 — same minute
Overcount — peak / average	9.1% / 90.1%
Serving latency p50 / p95 / p99	99 / 121 / 124 ms — 100 ms target, missed
Correctness	12 / 12 across five independent implementations
Sessions still open at file boundary	37,649 of 108,486 — 34.7%



Why the obvious answer is wrong

- The heartbeat is 40 seconds, not the 60 the data dictionary states.
p90 of exactly 40.0s across four telemetry streams. Every threshold derives from that, not the doc.
- There is no pause event type.
Pause, resume, speed-pause and AdPause all live inside the 41-value VideoHeartbeat bucket.
- Background and foreground markers do not pair.
14,700 AppBackgrounded vs 14,321 AppForegrounded — the model never waits for a match to close a period.



Filter, sum, then max — never max first

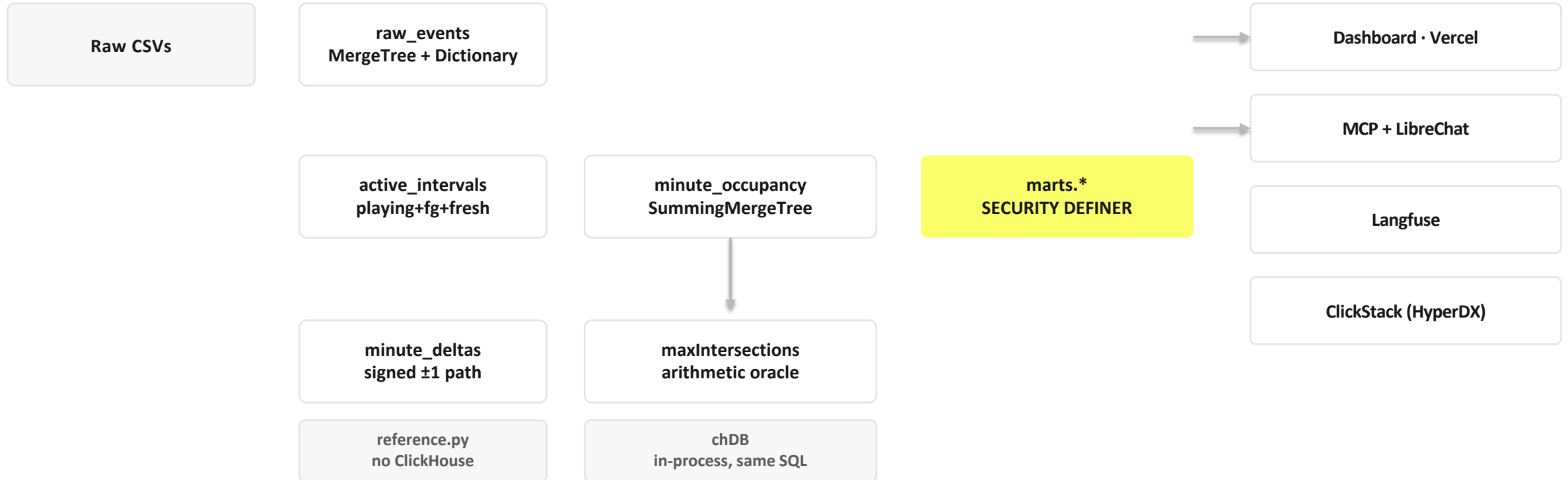
- Per-minute concurrency is additive across dimensions.

One deduped row per (session, minute) — one rollup answers every filter combination, at any grain.

- Peak is not composable across time.

`max()` doesn't distribute over sums, so per-slice peaks can't be added. Get the order wrong and the answer is silently wrong, not an error.

Everything the problem is made of runs in ClickHouse



Only the Python reference and chDB run outside the server — and chDB is still ClickHouse.



Five paths that share no code, then diffed

- `minute_occupancy` — the primary serving path
- `minute_deltas` — signed ± 1 runs, a windowed cumsum
- `maxIntersections` — arithmetic oracle, no rollup
- A Python reference — owes ClickHouse nothing
- `chDB` — same SQL files, in-process, no server

FOUR GATES DIFF THEM

- Gate A — 12 cross-path checks.
unfiltered, across every slice
- **Gate B — byte-identical rebuild.**
- Gate C — held-out day rehearsal.
caught a real async-mutation race
- Gate D — `chDB`, three runtimes.
same hash, three ClickHouse versions

12 / 12 — EXACTLY



Performance & scale

MEASURE	RESULT
Read cost, 0 → 8 dimension filters	97,043 → 99,968 rows — +2,925, not 8x more scans
Naive baseline (no rollup)	905,558 rows for the same answer
Projection pruning	136/180 → 26/136 granules; planner-chosen, not forced
Sharding at 100x	8 independent shards by session hash — exact, zero coordination

Every figure is `query_duration_ms` from `system.query_log` by `query_id` — never a client stopwatch.



The full stack, ClickHouse underneath every piece

CLICKHOUSE ITSELF

- ClickHouse Cloud — every stage, ingestion to serving
- Parameterized views + SQL SECURITY DEFINER
- SummingMergeTree, ReplacingMergeTree + MVs
- Dictionaries — content joined without a join
- Projections — planner-chosen, measured pruning
- chDB — same SQL, in-process, hash-identical
- ClickHouse managed Postgres — Langfuse's state
- readonly=1 CONST — the budget the DB enforces

EVERYTHING AROUND IT

- ClickStack, built on HyperDX — pipeline traces
- Langfuse — LLM & MCP traces, stored in ClickHouse
- LibreChat + Gemini 3 Flash — plain-English Q&A
- MCP — 5 custom tools + the official ClickHouse server
- Vercel — the dashboard, serverless, no admin creds
- 1 EC2 instance + Caddy — HTTPS via sslip.io
- Python, stdlib only — zero runtime dependencies
- OpenAI / Bedrock gpt-oss-120b — optional decline LLM



Live deployments and dashboards

clickliv.vercel.app

The concurrency dashboard — foreground-only vs naive, every dimension filterable live.

librechat.15-252-63-157.sslip.io

Ask for any published number in plain English — guardrailed tools, never raw SQL.

langfuse.15-252-63-157.sslip.io

LLM & MCP trace dashboard — traces, cost, tool calls, all live tiles.

clickstack.15-252-63-157.sslip.io

Pipeline trace dashboard, built on HyperDX — a span per stage and per query.



Update handling & the unseen day

- 34.7% of sessions are still open at the file boundary — real, not a fixture.
A ReplacingMergeTree + materialized view extends them live on every insert, with no rebuild.
- Proved the hard way: insert one heartbeat into an open session, read live state, then rebuild from scratch.
They agree to the millisecond.
- make preflight → make unseen → make rollback, 7.8 seconds to recover.
It held on the graded drop: two unseen columns, video_resolution and show_name, needed zero code changes.



Conclusions & honest limits

CONFIDENT ABOUT

- Correctness is diffed, not asserted — 12/12
- Read pattern is right; instance size is the bottleneck
- Generalizes to columns it never saw

NOT CLAIMING

- We miss our own 124 ms p99 target — stated plainly
- Occupancy and instantaneous are both computed
- Ambiguous titles refuse rather than guess

Impact: right-sized ad inventory, capacity provisioned against a real peak, a concurrency cliff visible within a minute.



Live dashboard

clickliv.vercel.app

Chat

librechat.15-252-63-157.sslip.io

LLM / MCP traces

langfuse.15-252-63-157.sslip.io

Pipeline traces

clickstack.15-252-63-157.sslip.io

Repository

github.com/ashutosh887/ClickLiv